

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Algorithm Design Challenge

COURSE NAME: ARTIFICIAL INTELLIGENCE **COURSE CODE:** MLA01
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO1: Apply search algorithms and heuristic techniques to solve state-space search and optimization problems. (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100

Time: 90 Mins

No. of Questions: 5

Annexure A

Algorithm Design Challenge

Code of Conduct:

I S.Venkateswarlu /192425340 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S.Venkateswarlu

Criterion	Weightage	Marks Obtained
Problem Analysis and Understanding	15%	
AI Algorithm Selection & Justification	15%	
Algorithm Design / Pseudocode	25%	
Application of AI Concepts (Greedy, A*, CSP, Minimax, etc.)	15%	
Diagram / State Space / Search Tree Representation	10%	
Complexity Analysis and Solution Efficiency	10%	
Originality, Critical Thinking, and Presentation	10%	
Total	100%	

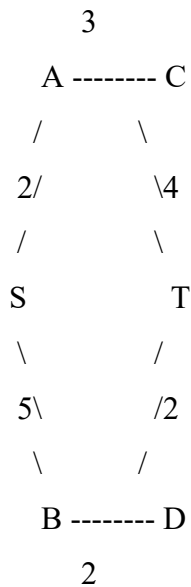
Signature of the Faculty

Total Marks Awarded

Q1. Greedy Best-First Search

1. Problem Analysis

Consider a navigation system that needs to find a route from **S (Start)** to **T (Target)**.



2. Heuristic Function

The heuristic estimates the remaining distance from the current node to the target.

$$f(n) = h(n)$$

Node	$h(n)$
S	6
A	5
B	4
C	2
D	1
T	0

3. Algorithm Selection

Greedy Best-First Search is selected because it chooses the node with the **lowest heuristic value**.

It focuses mainly on reaching the goal quickly.

4. Node Expansion

Step 1: Start at S.

Successors:

- $A \rightarrow h(A) = 5$
- $B \rightarrow h(B) = 4$

Select **B**.

Step 2: Expand B.

- $D \rightarrow h(D) = 1$

Select **D**.

Step 3: Expand D.

- $T \rightarrow h(T) = 0$

Select **T**.

5. Expansion Order

$S \rightarrow B \rightarrow D \rightarrow T$

6. Final Path

$S \rightarrow B \rightarrow D \rightarrow T$

7. Pseudocode

1. Create OPEN priority queue.
2. Insert Start into OPEN.
3. While OPEN is not empty:
 - a. Select node with minimum $h(n)$.
 - b. If node = Goal:
 - return path.
 - c. Expand the node.
 - d. Add unvisited successors to OPEN.
4. Return failure if OPEN becomes empty.

8. Advantages

1. Uses heuristic knowledge.
2. Can reach the goal quickly.
3. Simple to implement.
4. Useful when speed is more important than optimality.

9. Limitation

It does not guarantee the shortest path because it ignores the cost already travelled.

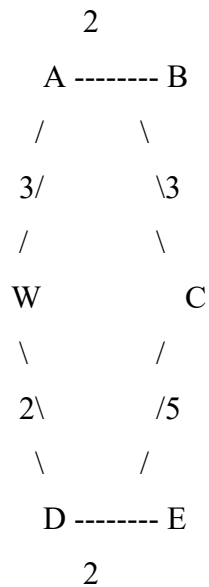
10. Complexity

- Time: $O(b^m)$
- Space: $O(b^m)$

Q2. A* Search for Delivery Robot

1. Problem Analysis

A delivery robot must travel from **W (Warehouse)** to **C (Customer)**.



The road numbers represent travel costs.

2. Heuristic Function

A* uses:

$$f(n) = g(n) + h(n)$$

Where:

1. $g(n)$ = actual cost from W to n.
2. $h(n)$ = estimated cost from n to C.
3. $f(n)$ = estimated total cost.

3. Heuristic Values

Node	$h(n)$
W	5
A	4
D	5
B	3
E	4
C	0

4. Calculation

For W:

$$g(W) = 0$$

$$h(W) = 5$$

$$f(W) = 0 + 5 = 5$$

For A:

$$g(A) = 3$$

$$h(A) = 4$$

$$f(A) = 7$$

For D:

$$g(D) = 2$$

$$h(D) = 5$$

$$f(D) = 7$$

For B:

$$g(B) = 3 + 2 = 5$$

$$h(B) = 3$$

$$f(B) = 8$$

For E:

$$g(E) = 2 + 2 = 4$$

$$h(E) = 4$$

$$f(E) = 8$$

For C through B:

$$g(C) = 5 + 3 = 8$$

$$h(C) = 0$$

$$f(C) = 8$$

5. Optimal Path

$$\mathbf{W \rightarrow A \rightarrow B \rightarrow C}$$

Total cost:

$$\mathbf{3 + 2 + 3 = 8}$$

6. Pseudocode

1. Put Start in OPEN.
2. Set $g(\text{Start}) = 0$.
3. Calculate $f(\text{Start}) = g(\text{Start}) + h(\text{Start})$.
4. Select the node with minimum $f(n)$.
5. If the node is Goal, return the path.
6. Expand the node.
7. Calculate g , h and f for successors.
8. Continue until Goal is reached.

7. Why A* is Optimal

A* guarantees an optimal solution when the heuristic is **admissible**, meaning that it never overestimates the actual remaining cost.

8. Advantages

1. Finds an optimal path.
2. Uses both actual and estimated costs.
3. More reliable than Greedy Best-First Search.
4. Useful in robot navigation and route planning.

9. Complexity

- Time: $O(b^d)$ worst case.
- Space: $O(b^d)$.

Q3. Examination Timetable Using CSP

1. Problem Analysis

A university wants to schedule examinations without placing conflicting subjects in the same time slot.

Subjects are:

1. AI
2. ML
3. DBMS
4. CN
5. OS

Available slots:

1. P1 – Monday Morning
2. P2 – Monday Afternoon
3. P3 – Tuesday Morning

2. Variables

Each subject is a variable.

$X = \{AI, ML, DBMS, CN, OS\}$

3. Domains

Each subject can be assigned:

$D = \{P1, P2, P3\}$

4. Constraints

Suppose the following subjects have common students:

1. $AI \neq ML$
2. $AI \neq DBMS$
3. $ML \neq OS$
4. $DBMS \neq CN$
5. $CN \neq OS$

Therefore, conflicting subjects cannot have the same time slot.

5. Constraint Graph



An edge represents a conflict.

6. MRV

MRV = Minimum Remaining Values.

MRV selects the variable having the fewest available values.

Example:

AI $\rightarrow$ P1, P2

ML $\rightarrow$ P1, P2, P3

DBMS $\rightarrow$ P2, P3

CN $\rightarrow$ P1, P2, P3

OS $\rightarrow$ P1, P3

AI, DBMS and OS have fewer choices.

MRV selects one of the most restricted variables first.

7. Forward Checking

Suppose:

AI = P1

Since AI conflicts with ML and DBMS:

ML $\rightarrow$ P2, P3

DBMS $\rightarrow$ P2, P3

P1 is removed from their domains.

8. Example Timetable

Subject	Slot
AI	P1
ML	P2

Subject	Slot
DBMS	P2
CN	P1
OS	P3

No conflicting subjects have the same slot.

9. Pseudocode

1. Select an unassigned variable using MRV.
2. Select a possible value.
3. Check constraints.
4. Assign the value.
5. Apply Forward Checking.
6. If a domain becomes empty, backtrack.
7. Continue until all variables are assigned.
8. Return the completed timetable.

10. Advantages of MRV

1. Selects difficult variables first.
2. Reduces unnecessary search.
3. Finds conflicts earlier.

11. Advantages of Forward Checking

1. Removes invalid future choices.
2. Detects conflicts early.
3. Reduces backtracking.

12. Complexity

Worst case:

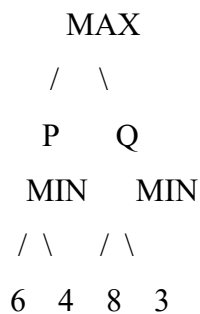
$O(d^n)$

where d is the number of domain values and n is the number of variables.

Q4. Minimax and Alpha-Beta Pruning

1. Problem Analysis

Consider the following game tree:



2. Minimax Calculation

For P:

$$P = \text{MIN}(6, 4)$$

$$P = 4$$

For Q:

$$Q = \text{MIN}(8, 3)$$

$$Q = 3$$

At MAX:

$$\text{MAX} = \text{MAX}(4, 3)$$

$$\text{MAX} = 4$$

3. Best Move

MAX chooses P.

Final decision:

$$\text{MAX} \rightarrow \text{P} \rightarrow 4$$

4. Alpha-Beta Pruning

Initially:

$$\alpha = -\infty$$

$$\beta = +\infty$$

After evaluating P:

$$\mathbf{P = 4}$$

Therefore:

$$\mathbf{\alpha = 4}$$

Now evaluate Q.

First value:

$$Q \rightarrow 8$$

Second value:

$$Q \rightarrow 3$$

Now:

$$\beta = 3$$

Since:

$$\beta \leq \alpha$$

$$3 \leq 4$$

any remaining children of Q can be pruned.

5. Pseudocode

1. Start with $\alpha = -\infty$ and $\beta = +\infty$.
2. If node is terminal, return its value.
3. For MAX:
 - Select the maximum child value.
 - Update α .
4. For MIN:
 - Select the minimum child value.
 - Update β .
5. If $\alpha \geq \beta$:
 - Prune the remaining branches.
6. Return the best value.

6. Node Evaluation

Method	Nodes Evaluated
--------	-----------------

Minimax	7
---------	---

Alpha-Beta	6
------------	---

Alpha-Beta evaluates fewer nodes while producing the same result.

7. Advantages

1. Reduces unnecessary computation.
2. Allows deeper game-tree exploration.
3. Gives the same result as Minimax.
4. Improves game-playing performance.

8. Complexity

Minimax:

$$O(b^d)$$

Alpha-Beta best case:

$$O(b^{(d/2)})$$

5. Warehouse Robot Using Online Search

1. Problem Analysis

A warehouse robot must move from a starting location to a destination.

The environment can change unexpectedly.

For example:

Initial:

S

.

.

. G

A new obstacle may appear:

$S \rightarrow \rightarrow X \rightarrow G$

↑

Obstacle

Therefore, a fixed route may no longer work.

2. Algorithm Selection

An **Online Search Agent** is suitable because it can make decisions while exploring the environment.

It follows:

Observe → Decide → Move → Learn → Re-plan

3. Working Process

1. Start from the current position.
2. Observe nearby locations.
3. Update the environment map.
4. Select the best available movement.
5. Move one step.
6. Check for new obstacles.
7. If the route is blocked, calculate another route.
8. Repeat until the destination is reached.

4. Example

Initial route:

$S \rightarrow A \rightarrow B \rightarrow C \rightarrow G$

Unexpected obstacle:

$S \rightarrow A \rightarrow X \rightarrow C \rightarrow G$

↑

Blocked

The robot detects the obstacle and changes the route:

$S \rightarrow A \rightarrow D \rightarrow E \rightarrow C \rightarrow G$

5. Pseudocode

ONLINE_SEARCH(Start, Goal)

1. Current $\leftarrow$ Start.
2. Create an initially known map.
3. While Current $\neq$ Goal:
 - a. Observe the environment.
 - b. Update the map.
 - c. Check whether the current path is blocked.
 - d. If blocked, calculate a new route.
 - e. Select the best action.
 - f. Execute the action.
 - g. Update Current.
4. Stop when Goal is reached.

6. Why Online Search is Suitable

1. Handles unknown environments.
2. Detects new obstacles.
3. Supports real-time replanning.
4. Does not require a complete map.
5. Suitable for autonomous robots.

7. Comparison

Feature	Greedy BFS	A*	Online Search
Known environment	Yes	Yes	Not necessary
Dynamic obstacles	Poor	Limited	Excellent
Replanning	No	Limited	Yes
Heuristic	Yes	Yes	Can use

Feature	Greedy BFS	A*	Online Search
Optimality	No	Yes*	Depends on strategy
Adaptability	Low	Medium	High

8. Complexity

Online search does not have one fixed complexity because it depends on:

1. Environment size.
2. Number of obstacles.
3. Number of states explored.
4. Frequency of environmental changes.

9. Overall Comparison of the Five AI Techniques

No.	Problem	Algorithm	Main Concept
1	Route finding	Greedy Best-First Search	Heuristic-based search
2	Delivery robot	A* Search	Cost + heuristic
3	Exam timetable	CSP	Variables, domains, constraints
4	Game playing	Minimax + Alpha-Beta	Adversarial search
5	Dynamic warehouse	Online Search	Real-time adaptation

10. Conclusion

1. **Greedy Best-First Search** is useful for fast goal-directed search.
2. **A* Search** is suitable when the optimal path is required.
3. **CSP with MRV and Forward Checking** efficiently solves scheduling problems.
4. **Minimax with Alpha-Beta Pruning** improves decision-making in competitive games.
5. **Online Search** is suitable for robots working in changing and unpredictable environments.